

DAEDALUS

Desktop Client — JavaFX + Spring Boot Reference

:: Thin JavaFX shell that embeds the full Spring server + plugin runtime ::

Module: daedalus-desktop

Main Class: com.daedalus.desktop.DaedalusLauncher

Key Innovation: Boots Spring **headlessly first**, then hands control to JavaFX.

Pluggable Themes: Any plugin can contribute a Theme bean + CSS — appears automatically in the UI.

1. Boot Sequence (Critical Order)

The desktop module deliberately boots Spring **before** JavaFX so that all beans (including plugin-contributed ones) are available when the UI is constructed. This is the reverse of the more common "JavaFX first" pattern.

```
DaedalusLauncher.main()
    ■■■ SpringApplication.builder(DaedalusApp.class)
        .headless(false)
        .run(args) // Full Spring context (server + plugins + themes)
    ■■■ Application.launch(DaedalusPrimaryStage.class)
        ■■■ init() → DaedalusLauncher.springContext()
        ■■■ start(stage)
            ■■■ FXMLLoader + controllerFactory = spring::getBean
            ■■■ ThemeManager.applyDefault(scene)
            ■■■ stage.show()
```

1.1 Shutdown Contract

Both `stage.setOnCloseRequest` and `Application.stop()` call `DaedalusLauncher.shutdown()`, which safely closes the Spring context. The test `DaedalusLauncherTest` proves `shutdown()` is a no-op when the context was never created.

2. DaedalusPrimaryStage — Spring-Aware JavaFX

JavaFX instantiates this class. It pulls the already-booted Spring context and uses it to wire FXML controllers via a custom `controllerFactory`. This is the only place the desktop module touches Spring directly.

```
@Override
public void init() {
    this.spring = DaedalusLauncher.springContext();
    this.themeManager = spring.getBean(ThemeManager.class);
}

@Override
public void start(Stage stage) {
    FXMLLoader loader = new FXMLLoader(fxmlUrl);
    loader.setControllerFactory(spring::getBean);    // ← Magic: @Autowired works in FXML controllers
    Parent root = loader.load();
    Scene scene = new Scene(root, 1280, 800);
    themeManager.applyDefault(scene);
    stage.setScene(scene);
    stage.setOnCloseRequest(e -> DaedalusLauncher.shutdown());
    stage.show();
}
```

2.1 FXML Controller Injection

Any FXML controller can now use `@Autowired` (or constructor injection) because the `controllerFactory` delegates to Spring's `getBean()`. This gives full access to `MazeGenerationService`, `PluginManager`, `ThemeManager`, STOMP clients, etc. from the UI layer.

3. Pluggable Theme System (Plugin-Extensible)

Themes are first-class Spring beans. Any plugin JAR that registers a `Theme` implementation + provides a CSS stylesheet on the classpath will appear automatically in the UI without any desktop code changes.

3.1 Theme Interface

```
public interface Theme {
    String id(); // "cosmic", "noir", "synthwave"...
    String displayName();
    String stylesheetPath(); // "/ui/cosmic.css" (classpath resource)
    Color passage(), wall(), start(), goal(), path(), visited(),
        frontier(), player(), background(), accent();
}
```

3.2 ThemeManager (Spring @Component)

```
@Component
public class ThemeManager {
    private final Map<String, Theme> themes = new ConcurrentHashMap<>();
    private Theme active;

    public ThemeManager(List<Theme> all, @Value("${daedalus.ui.theme:cosmic}") String defaultId) {
        for (Theme t : all) themes.put(t.id(), t);
        this.active = themes.getDefault(defaultId, all.isEmpty() ? null : all.get(0));
    }

    public void apply(Scene scene, String id) { ... }
    public void applyDefault(Scene scene) { apply(scene, defaultId); }
}
```

Test coverage: `ThemeManagerTest` exercises the three constructor branches (named default present, missing → fallback, no themes).

4. Module Dependency Graph

Module	Depends On	Provides
daedalus-desktop	daedalus-server	JavaFX launcher + ThemeManager + FXML shell
daedalus-server	daedalus-plugin-runtime + daedalus-core	REST + STOMP + PluginManager + services
daedalus-plugin-runtime	daedalus-plugin-api + daedalus-core	Plugin lifecycle, JAR discovery, events
daedalus-core	—	MazeGrid, generators, solvers, models
daedalus-plugin-api	—	MazePlugin SPI, PluginManifest, events (Spring-free)

The desktop module is completely optional. You can run `DaedalusApp` directly for a headless server, or embed `daedalus-plugin-runtime` in any other Spring application.

This completes the full Daedalus stack reference. You now have PDFs for: Server Configs & Audit Fixes, Plugin Runtime Architecture, and Desktop Client. All TypeScript DTOs are in `daedalus-api-dtos.ts`.